

PENGUJIAN APLIKASI MENGGUNAKAN *WHITE BOX* DENGAN METODE *UNIT TESTING*. STUDI KASUS: MAHATI

Muhammad Rizky Haksono¹⁾, Ilyas Nuryasin²⁾, Bashor Fauzan Muthohirin^{*3)}, Mutiara Titani⁴⁾, Akrom⁵⁾, Abdul Fadlil⁶⁾

1. Informatika, Teknik, Universitas Muhammadiyah Malang, Indonesia
2. Informatika, Teknik, Universitas Muhammadiyah Malang, Indonesia
3. Informatika, Teknik, Universitas Muhammadiyah Malang, Indonesia
4. Farmasi, Ilmu Kesehatan, Universitas Muhammadiyah Malang, Indonesia
5. Fakultas Farmasi, Universitas Ahmad Dahlan, Indonesia
6. Fakultas Teknologi Industri, Universitas Ahmad Dahlan, Indonesia

Article Info

Kata Kunci: Aplikasi Backend; Pengujian; White Box Testing; Unit Testing; Branch Coverage; Statement Coverage;

Keywords: Backend Application; Testing; White Box Testing; Unit Testing; Branch Coverage; Statement Coverage

Article history:

Received 27 November 2024

Revised 12 December 2024

Accepted 17 December 2024

Available online 1 December 2025

DOI :

<https://doi.org/10.29100/jipi.v10i4.7012>

* Corresponding author.

Bashor Fauzan Muthohirin

E-mail address:

bashorfauzan@umm.ac.id

ABSTRAK

Perkembangan pesat industri perangkat lunak secara signifikan dipengaruhi oleh kemajuan teknologi informasi dan digital. Salah satu aspek penting dalam perkembangan ini adalah peningkatan efisiensi pengujian perangkat lunak. Pengujian perangkat lunak saat ini tidak hanya berfokus pada aspek fungsionalitas, tetapi juga mencakup kinerja, keamanan, dan kompatibilitas. Penelitian ini berfokus pada peningkatan efisiensi pengujian perangkat lunak dengan menerapkan metode *White-Box Testing*. Penerapannya diuji pada aplikasi MAHATI melalui unit testing dengan teknik *Branch Coverage* dan *Statement Coverage* untuk memastikan cakupan fungsionalitas dan logika program secara komprehensif. Penelitian ini menekankan desain kode yang terstruktur dan mudah diuji, sehingga proses pengujian menjadi lebih efisien dan efektif dalam mendeteksi kesalahan. Hasil penelitian menunjukkan bahwa desain kode yang baik secara signifikan berkontribusi terhadap kualitas dan keandalan perangkat lunak melalui pengujian yang optimal dan efisien. Keberhasilan seluruh 38 *suite* pengujian dan 157 tes individu membuktikan bahwa setiap fungsi aplikasi bekerja sesuai dengan harapan, sehingga menunjukkan efektivitas metode pengujian yang diterapkan. Hasil ini menguatkan keyakinan bahwa *White-Box Testing* merupakan metode yang efektif dalam meningkatkan kualitas perangkat lunak dan dapat diaplikasikan untuk meningkatkan kualitas aplikasi lain.

ABSTRACT

The rapid advancement of the software industry is significantly influenced by progress in information and digital technology. A crucial aspect of this evolution is the enhancement of software testing efficiency. Modern software testing goes beyond functional aspects to include performance, security, and compatibility. This research focuses on improving software testing efficiency by employing the White-Box Testing method. Its implementation is tested on the MAHATI application through unit testing using Branch Coverage and Statement Coverage techniques to ensure comprehensive evaluation of the program's functions and logic paths. The study emphasizes code design with a focus on improving testability, making unit testing more straightforward and effective in identifying potential errors. The results demonstrate that effective code design significantly contributes to enhancing the quality and reliability of software through thorough and efficient testing methods. The success of all 38 suite tests and 157 individual tests passing without any failures indicates that each function of the application is operating as expected, highlighting the effectiveness of the applied testing approach. These findings reinforce the belief that White-Box Testing is an effective method for improving software quality and can be applied to enhance the quality of other applications.

I. PENDAHULUAN

Perkembangan industri *software* saat ini dipengaruhi secara signifikan oleh pesatnya perkembangan teknologi informasi dan digital. Salah satu aspek penting dari transformasi digital adalah penerapan pengujian *software testing* yang lebih efektif dan efisien, terutama dalam menghadapi tuntutan kualitas dan kecepatan. Pengujian perangkat lunak tidak hanya fokus pada perilaku fungsional; itu juga memeriksa detail implementasi internal melalui pengujian struktural, memastikan jaminan kualitas komprehensif di berbagai komponen perangkat lunak [1]. Pengujian perangkat lunak, yang sering disebut di bidang rekayasa perangkat lunak dengan singkatan yang umum digunakan dari “pengujian,” mewakili aktivitas kritis dan sangat diperlukan yang memainkan peran penting dalam memastikan kualitas dan fungsionalitas keseluruhan sistem perangkat lunak. Proses pengujian memerlukan eksekusi sistematis aplikasi perangkat lunak dengan tujuan eksplisit memverifikasi dan memvalidasi pengiriman data dengan cermat terhadap persyaratan yang telah ditetapkan dan ditentukan yang telah ditetapkan selama siklus pengembangan perangkat lunak [2], [3]. Secara langsung, prosedur pengujian perangkat lunak akan secara sistematis menganalisis dan menjalankan program untuk mengidentifikasi perbedaan.

Proses pengujian *software* melibatkan analisis dan pengukuran ketidaksesuaian antara model proses dan eksekusi, menggunakan teknik validasi proses untuk mengidentifikasi dan mengukur masalah yang muncul [4]. Untuk mengatasi perbedaan dan untuk mencegah kejadian yang tidak diinginkan dalam perangkat lunak, *white box testing* digunakan sebagai pendekatan metodologis untuk evaluasi perangkat lunak. Strategi yang terkait dengan *white box testing* mencakup perumusan tes untuk memastikan bahwa setiap baris kode dijalankan setidaknya sekali, atau bahwa setiap fungsi tunduk pada pengujian independen. Pengujian *white box* merupakan teknik penilaian perangkat lunak yang memerlukan analisis arsitektur internal atau mekanisme program, sehingga memfasilitasi cakupan yang lengkap dan peningkatan proses pengembangan pengujian. [5]. Pengujian dengan cara ini akan akurat bila penguji benar-benar mengerti apa yang harus dilakukan oleh program [6].

Para pengembang mencari cara untuk mengurangi waktu dan biaya pengembangan aplikasi. Mereka juga ingin membuat aplikasi *REST API* yang efektif, sederhana, dan sesuai dengan berbagai platform [7]. Salah satu pesaing yang paling menonjol adalah *Express.js*. *Express.js* adalah kerangka kerja yang meningkatkan produktivitas bagi para insinyur perangkat lunak dengan memungkinkan hasil yang lebih cepat dengan kualitas yang lebih baik melalui penggunaan dan pemeliharaan yang luas oleh pengembang dan kontributor [8], [9]. Fitur inti dari *Express.js* sangat mendukung untuk pengujian perangkat lunak dikarenakan efektivitas dan pemilihan *library* dalam javascript sangat luas [10]. Maka dari itu dipilihlah menggunakan *jest* sebagai *library* pendukung untuk menguji perangkat lunak MAHATI. Dalam *Express.js* memiliki dukungan *middleware* yang dapat memungkinkan penggunaan fungsi *middleware* untuk menangani *request*, *response*, dan *errors* [11] [12]. *Routing* dalam *Express.js* menyediakan mekanisme perutean yang kuat yang memungkinkan pengembang untuk menentukan rute untuk metode HTTP dan URL yang berbeda, sehingga mudah untuk mengelola *endpoint* [13].

Aplikasi memiliki dua arsitektur secara umum, *frontend* dan *backend*. *Backend* adalah komponen sisi *server* dari aplikasi yang menyimpan, memproses, menampilkan data untuk pengguna, mengaktifkan fungsi seperti retensi data, notifikasi, dan pengiriman konten [14], [15]. *Backend* bertanggung jawab untuk menerapkan logika bisnis, memastikan akses informasi yang cepat, dan penyimpanan data yang aman, serta melengkapi interaksi pengguna *frontend* dalam pengembangan situs web [16]. Aplikasi *backend* umum yang saat ini beroperasi sering mengalami kekurangan signifikan dalam kapasitasnya untuk menyelaraskan dan sesuai dengan *output* yang dihasilkan.

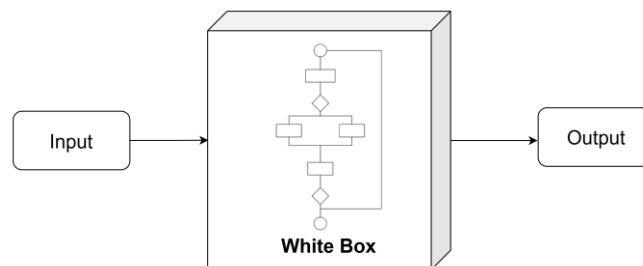
Ketidakpastian *output* dalam pengembangan perangkat lunak muncul dari durasi tugas yang tidak tepat, kesalahan pengkodean, dan variabilitas dalam keterampilan dan tingkat pembelajaran anggota tim, yang mempengaruhi penjadwalan dan perencanaan proyek [17]. Ketidaksesuaian antara ekspektasi dan hasil aplikasi juga tidak hanya mengganggu interaksi pengguna, tetapi juga berpotensi menyebabkan kesalahan dalam input data. Masalah ketidakakuratan data ini dapat berujung pada *output* yang gagal atau tidak valid, sehingga meningkatkan kemungkinan pengguna beralih ke aplikasi lain sebagai alternatif [18].

Solusi untuk mengatasi permasalahan ketidakpastian pada *output* aplikasi, permasalahan pengujian manual dan menciptakan lingkungan pengujian yang lebih baik pada para developer yaitu dengan menggunakan pengujian *software* seperti *unit testing*. *Unit testing* adalah menulis tes kecil yang menargetkan perilaku fungsi, biasanya dilakukan oleh pengembang, bukan *Quality Assurance*, untuk memastikan kualitas kode dan fungsionalitas pada tingkat granular [19]. Pada penelitian ini akan menerapkan metode yang berfokus pada kode dalam aplikasi yaitu *White-Box Testing* dengan pengujian menggunakan *unit testing*. Kelebihan metode ini yaitu menguji keterlibatan fungsi pada kode aplikasi yang mengidentifikasi jalur kode internal dan dependensi, memfasilitasi deteksi *bug* awal dan meningkatkan pemeliharaan melalui pengujian sistematis program yang kompleks [20].

Beberapa penelitian sebelumnya dengan studi kasus yang sama dengan tujuan untuk mempermudah proses pengujian memiliki kelemahan yaitu tidak dapat digeneralisasikan untuk semua kasus, karena penelitian ini tidak

secara ekstensif mengeksplorasi implikasi peningkatan pada pemeliharaan jangka panjang dari tes yang dihasilkan. Pengujian yang dilakukan untuk aplikasi itu menggunakan *White Box Testing* dan hasil pengujian ini melaporkan peningkatan sistematis dalam branch coverage dengan peningkatan rata-rata sekitar 1,5% [21]. Penelitian serupa dengan metode yang sama yaitu unit testing dengan pendekatan metode analisis dengan tujuan mengungkapkan bahwa 61% dari semua tes mengandung lebih dari satu pernyataan, yang bertentangan dengan praktik terbaik yang merekomendasikan satu pernyataan per tes. Para peneliti tidak sepenuhnya mewakili praktik pengujian Java yang lebih luas [22]. Penelitian lain dengan metode yang sama menggunakan metode branch coverage untuk pengujian secara otomatis dengan menguji lebih dari 3000 *dataset* kelas Java. Penelitian ini berfokus pada serangkaian fitur yang terbatas, dan tidak mengeksplorasi dari *method* lain yang mungkin dapat meningkatkan akurasi prediksi [23]. Dari penelitian yang sudah dijabarkan, kontribusi dari penelitian ini adalah untuk meningkatkan kualitas dan melihat berbagai kemungkinan kesalahan yang ada pada kode secara komprehensif. Selain itu dapat mendeteksi metrik kualitas kode dalam *statement*, *branch*, *functions*, *line*, dan *unused lines*.

II. METODE PENELITIAN



Gambar. 1. Model Tahapan White-Box Testing

Dalam penelitian ini dibuat sebuah aplikasi MAHATI yang di dalamnya menerapkan arsitektur 3 *tier*, pola *Model View Controller* (MVC). Program yang dikembangkan bertujuan untuk memberikan model terkait penggunaan metode pengujian *unit testing* menggunakan kerangka kerja pengujian otomatis Jest dengan bahasa Javascript [24]. Penelitian ini berfokus pada pengujian perangkat lunak dengan metode pengujian *unit*, sehingga program yang dibangun hanya mencakup entitas yang relevan dan diperlukan dalam pengujian. Penelitian ini menerapkan metode *White-Box Testing* untuk menguji aplikasi MAHATI. Metode *White-Box Testing* pada penelitian ini memiliki 2 teknik pengujian yaitu *Branch Coverage*, dan *Statement Coverage* [25]. Model tahapan metode ini bisa diamati melalui Gambar 1.

A. Branch Coverage

Pada tahap ini *Branch Coverage* adalah metode pengujian yang bertujuan untuk memaksimalkan jumlah cabang program yang dikunjungi oleh serangkaian kasus uji, terutama pada kondisional dan perulangan dalam kode program [26] [27]. Pengujian ini mengukur rasio jumlah cabang yang berbeda (menghitung cabang yang benar dan salah dalam pernyataan atau ekspresi) yang dieksekusi oleh rangkaian uji coba terhadap jumlah total cabang dalam program. Penilaian ini secara konsisten mengevaluasi dengan menghasilkan minimal dua tes data untuk membuktikan setiap kondisi yang berkaitan dengan kebenaran dan kepalsuan pernyataan fungsi [28]. *Branch coverage* adalah metrik yang lebih kuat daripada *line* dan *statement coverage*; namun *line* atau *statement coverage* tidak dapat menjamin *branch coverage* 100%, tetapi *branch coverage* 100% memberikan cakupan *line* dan *statement* 100% [29] [30].

Branch coverage sebagaimana diilustrasikan dalam kerangka Gambar 2, memenuhi fungsi penting dalam memastikan bahwa setiap cabang bersyarat dalam suatu fungsi telah menjalani pengujian komprehensif untuk memvalidasi keandalan fungsionalnya. Setiap kondisi individu yang muncul dalam struktur pernyataan *if* dan *else if* mengharuskan pembuatan minimal dua kasus uji yang berbeda: kasus pertama dimaksudkan untuk mengevaluasi skenario di mana kondisi dievaluasi menjadi benar, sedangkan kasus kedua dirancang untuk menilai skenario alternatif di mana kondisi dievaluasi salah. Pendekatan ini berperan penting dalam memvalidasi bahwa fungsi bereaksi dengan cara yang sesuai dan diharapkan ketika dikenakan beragam nilai input, sehingga meningkatkan keandalan kode secara keseluruhan.

1. Kondisi *Sistole*:

- Kasus Uji untuk *true*: Input sistol kurang dari 20 atau lebih dari atau sama dengan 200. Harus menghasilkan respons dengan pesan kesalahan yang menjelaskan batas valid untuk sistol.
- Kasus Uji untuk *false*: Input sistol antara 20 dan 199. Fungsi harus melanjutkan untuk memeriksa kondisi berikutnya.

2. Kondisi *Diastole*:

- a. Kasus Uji untuk *true*: Input diastole kurang dari 20 atau lebih dari atau sama dengan 200. Harus menghasilkan respons dengan pesan kesalahan yang menjelaskan batas valid untuk diastole.
 - b. Kasus Uji untuk *false*: Input diastole antara 20 dan 199. Fungsi harus melanjutkan untuk memeriksa kondisi berikutnya.
3. Kondisi *Heartbeat*:
- a. Kasus Uji untuk *true*: Input heartbeat kurang dari 20 atau lebih dari atau sama dengan 200. Harus menghasilkan respons dengan pesan kesalahan yang menjelaskan batas valid untuk denyut jantung.
 - b. Kasus Uji untuk *false*: Input heartbeat antara 20 dan 199. Fungsi harus melanjutkan untuk memeriksa kondisi berikutnya.
4. Kondisi Format Angka:
- a. Kasus Uji untuk *true*: Input berisi karakter non-numerik seperti ",", atau ".". Harus menghasilkan respons dengan pesan yang meminta penggunaan bilangan bulat.
 - b. Kasus Uji untuk *false*: Input hanya mengandung angka tanpa karakter khusus. Fungsi harus berhasil menambahkan data tekanan darah.

Setelah semua validasi berhasil, data tekanan darah baru dibuat dalam *database* menggunakan Prisma ORM. Data yang disimpan meliputi *user_id* (diambil dari *token* yang diverifikasi), *image* (jika ada), serta nilai *sistol*, *diastol*, dan *heartbeat* yang sudah dikonversi menjadi *integer*. Jika proses pembuatan data dijalankan dengan sukses, fungsi akan menghasilkan respons yang berhasil ditandai dengan kode status 201, disertai dengan data tekanan darah yang baru dibuat. Sebaliknya, jika kesalahan muncul selama proses validasi atau penyimpanan data, kesalahan akan ditransmisikan ke *middleware* untuk penanganan kesalahan yang tepat melalui pemanggilan berikutnya *next(error)*.

```
export const createBloodPressure = async (req, res, next) => {
  try {
    const { systol, diastole, heartbeat } = req.body;
    const data = verifyToken(req.headers.access_token);
    if (data?.status) return res.status(data.status).json(data);

    const image = req.file ? req.file.path : "";

    if (parseInt(systol) < 20 || parseInt(systol) >= 200) {
      return res.status(400).json({
        status: 400,
        message: "Sistol value should be between 20 and 200.",
      });
    } else if (parseInt(diastole) < 20 || parseInt(diastole) >= 200) {
      return res.status(400).json({
        status: 400,
        message: "Diastole value should be between 20 and 200.",
      });
    } else if (parseInt(heartbeat) < 20 || parseInt(heartbeat) >= 200) {
      return res.status(400).json({
        status: 400,
        message: "Heartbeat value should be between 20 and 200.",
      });
    } else if (
      systol.includes(",") ||
      diastole.includes(",") ||
      heartbeat.includes(",") ||
      systol.includes(".") ||
      diastole.includes(".") ||
      heartbeat.includes(".")
    ) {
      return res.status(400).json({
        status: 400,
        message: "Please use whole numbers for blood pressure values.",
      });
    }
  } catch (error) {
    next(error);
  }
}
```

```
const newBloodPressure = await prisma.blood_pressures.create({
  data: {
    user_id: data.id,
    image,
    systol: parseInt(systol),
    diastole: parseInt(diastole),
    heartbeat: parseInt(heartbeat),
  },
});
res.status(201).json({ success: true, data: newBloodPressure });
```

Gambar. 2. Penerapan Branch Coverage

B. Statement Coverage

Statement Coverage adalah teknik pengujian yang digunakan dalam pengembangan perangkat lunak untuk memastikan bahwa setiap pernyataan dalam kode sumber dieksekusi sedikitnya sekali selama proses pengujian

[27]. Tujuannya adalah untuk mengidentifikasi bagian kode yang tidak tercapai oleh pengujian normal, yang mungkin mengindikasikan adanya *bug* atau kekurangan dalam pengujian tersebut. [25]. Dari Gambar 3 merupakan contoh fungsi *createBloodPressure* yang bertujuan untuk mengolah data *sistole*, *diastole*, dan *heartbeat* yang diterima melalui HTTP *request*. Angka "12x" di baris 4 yang terlihat dalam warna hijau pada Gambar 3 menunjukkan jumlah kali baris kode tersebut telah dieksekusi selama proses pengujian. Dalam konteks pengujian perangkat lunak, angka ini menunjukkan seberapa sering baris tertentu telah dilalui atau dijalankan selama sesi pengujian yang telah dilakukan.

Jika frekuensi eksekusi pada pernyataan validasi rendah, ini mungkin menunjukkan bahwa hanya input valid yang diuji, sementara jalur validasi kesalahan belum teruji secara memadai. Dengan teknik *Statement Coverage*, pengembang dapat mengevaluasi dan memperbaiki pengujian agar lebih komprehensif, sehingga seluruh jalur logika pada fungsi *createBloodPressure* benar-benar diuji dan risiko *bug* dapat diminimalisir.

```
4 12x export const createBloodPressure = async (req, res, next) => {  
5 15x   try {  
6 15x     const { sistol, diastole, heartbeat } = req.body;  
7 15x     const data = verifyToken(req.headers.access_token);  
8 15x     if (data?.status) return res.status(data.status).json(data);  
9  
10 13x     const image = req.file ? req.file.path : "";  
11  
12 13x     if (parseInt(sistol) < 20 || parseInt(sistol) >= 200) {  
13 5x       return res.status(400).json({  
14         status: 400,  
15         message: "Sistol value should be between 90 and 200.",  
16       });  
17 8x     } else if (parseInt(diastole) < 20 || parseInt(diastole) >= 200) {  
18 2x       return res.status(400).json({  
19         status: 400,  
20         message: "Diastole value should be between 60 and 200.",  
21       });  
22 6x     } else if (parseInt(heartbeat) < 20 || parseInt(heartbeat) >= 200) {  
23 2x       return res.status(400).json({  
24         status: 400,  
25         message: "Heartbeat value should be between 60 and 200.",  
26       });  
27 4x     } else if (  
28       sistol.includes(",") ||  
29       diastole.includes(",") ||  
30       heartbeat.includes(",") ||  
31       sistol.includes(".") ||  
32       diastole.includes(".") ||  
33       heartbeat.includes(".")  
34     ) {  
35 1x       return res.status(400).json({  
36         status: 400,  
37         message: "Please use whole numbers for blood pressure values.",  
38       });  
39     }  
}
```

Gambar. 3. Penerapan Statement Coverage

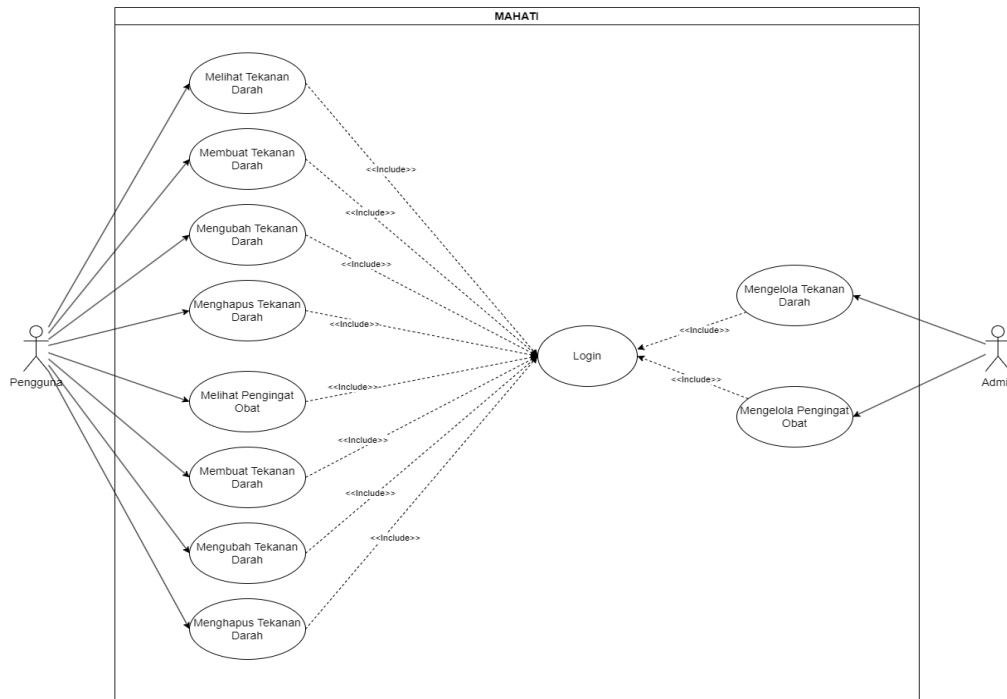
C. Perancangan Diagram *Use Case*

Diagram *Use Case* adalah representasi visual yang membantu pengembang perangkat lunak dalam memahami perilaku sistem, diekstraksi melalui rekayasa balik untuk meningkatkan pemeliharaan dan pembaruan sistem [31]. *Use Case* menggambarkan keterlibatan antara berbagai aktor dan sistem. *Use case* mempresentasikan fungsi-fungsi tertentu, termasuk tetapi tidak terbatas pada, otentikasi sistem, memasukkan informasi terkait hipertensi ke dalam sistem, di antara kegiatan lainnya. Gambar 4 memperlihatkan berbagai interaksi yang mungkin terjadi antara pengguna dan sistem, serta peran administrator dalam mengelola konten dan fungsi sistem. Pengguna memiliki kemampuan untuk mengelola data pribadi terkait kesehatan mereka, seperti mencatat, melihat, mengubah, dan menghapus data tekanan darah, serta mengelola pengingat obat. Sebelum melakukan aktivitas tersebut, pengguna harus terlebih dahulu melakukan *login*, yang menjadi langkah awal dan prasyarat untuk mengakses fitur-fitur sistem. Setelah *login*, pengguna dapat melihat riwayat tekanan darah mereka, menambahkan data baru, memperbarui informasi yang sudah ada, atau menghapus data yang tidak diperlukan. Selain itu, pengguna juga dapat mengakses fitur pengingat obat untuk menjadwalkan waktu pengobatan, memastikan tidak ada dosis yang terlewat.

Sementara itu, administrator memiliki akses yang lebih luas dibandingkan pengguna biasa. Admin bertanggung jawab untuk mengelola seluruh data dalam sistem, termasuk data tekanan darah dan pengingat obat milik semua pengguna. Fungsi ini mencakup kemampuan untuk menambahkan, mengubah, dan menghapus data secara kolektif, serta memastikan integritas dan keakuratan data yang tersimpan. Admin juga berperan dalam menjaga kelancaran operasional sistem dengan mengelola konten edukasi dan memberikan dukungan kepada pengguna yang membutuhkan.

Semua fungsi dalam sistem dihubungkan oleh proses *login* yang menjadi pusat interaksi antara aktor dan sistem. Hubungan *<<include>>* yang ditampilkan pada diagram menunjukkan bahwa *login* diperlukan untuk semua

aktivitas, baik yang dilakukan oleh pengguna maupun admin. Sistem MAHATI dirancang untuk mempermudah pengguna dalam memantau dan mengelola kesehatan mereka, khususnya terkait tekanan darah dan pengingat obat, sekaligus memberikan administrator sumber daya yang mereka perlukan untuk mendukung dan menjaga sistem berfungsi secara keseluruhan.



Gambar. 4. Use Case Mahati

Selain itu, kerangka kerja Express.js yang digunakan oleh aplikasi MAHATI melakukan pendekatan *White Box* digunakan untuk pengujian. *White Box Testing* berkonsentrasi pada kode sumber perangkat lunak. Tujuan dari pengujian adalah untuk menemukan kekurangan atau cacat yang mungkin muncul pada antarmuka pengguna, struktur data, atau fungsi ketika program dijalankan. Semua komponen program perangkat lunak, termasuk data internal, perulangan, logika keputusan, dan jalur logika program, diuji dengan menggunakan data uji. Untuk menjamin cakupan pengujian yang komprehensif, dua metodologi *white box testing* – *statement coverage* dan *branch coverage* – diterapkan dalam pengujian ini.

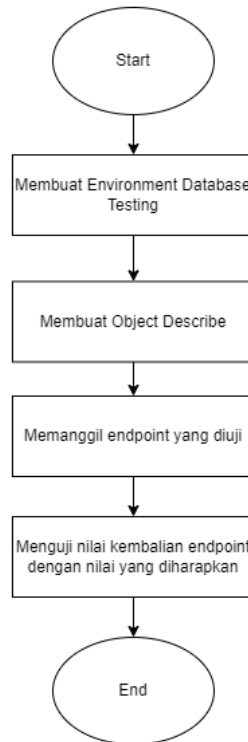
D. Perancangan Testabilitas Kode

Tidak semua kode perangkat lunak dapat diuji dengan mudah menggunakan pengujian unit. Oleh karena itu, diperlukan perancangan yang tepat dalam penulisan kode agar kode tersebut lebih mudah diuji melalui pengujian *unit*. Berikut adalah perancangan penulisan kode yang dilakukan penulis untuk mencapai tingkat testabilitas kode yang optimal.

1. Menerapkan *function*, dengan menggunakan *function* maka pada proses *mocking* (memalsukan *object* yang menjadi fungsi yang sedang diuji) akan mudah dilakukan.
2. Menerapkan *branch coverage testing*, dengan menggunakan *function testing* maka pada proses perkondisian dan perulangan kode jadi lebih mudah untuk diketahui jika ada kesalahan logika.
3. Menerapkan *statement coverage testing*, maka setiap *function* dan *method* yang ada di dalam kode akan dieksekusi sedikitnya sekali untuk melihat apakah setiap pernyataan mencakup setidaknya satu kali *test*.

E. Perancangan Pengujian Unit

Gambar 5 menggambarkan langkah-langkah dalam pengujian unit untuk aplikasi MAHATI. Proses ini diawali dengan persiapan lingkungan pengujian yang melibatkan pengaturan *database* khusus untuk testing, yang diisi dengan data buatan. Langkah selanjutnya adalah mendefinisikan objek yang akan diuji, dengan menentukan *input* dan *output* yang diharapkan. Setelah objek tersebut diuji, hasil yang didapatkan kemudian dibandingkan dengan hasil yang diharapkan. Jika hasil tes sesuai dengan prediksi, maka objek tersebut dianggap memenuhi kriteria pengujian. Proses ini ditutup dengan pencatatan hasil untuk tujuan dokumentasi dan analisis lebih lanjut.



Gambar.5. Diagram Alir Pengujian Unit

F. Perancangan Pengujian Unit pada Peningkat Obat dan Tekanan Darah Service

Use case pada Tabel I menjelaskan situasi di mana pengguna yang sudah *login* menggunakan *access token* bisa melihat data tentang pengingat obat dan tekanan darah mereka. Prosesnya dimulai saat pengguna masuk ke *endpoint* khusus dengan *access token* yang berlaku. Sistem kemudian mengambil data yang dibutuhkan dan menampilkan informasi terbaru kepada pengguna. Informasi yang baru ini akan ditampilkan di bagian awal agar pengguna selalu mendapatkan data terkini.

TABEL I
 DESKRIPSI USE CASE DIAGRAM MELIHAT PENGINGAT OBAT DAN TEKanan DARAH

UC Name	Melihat Peningkat Obat & Tekanan Darah	
Actor	Pengguna	
Goal	Pengguna dapat melihat data pengguna	
Pre-Condition	1. Pengguna belum punya <i>access token</i>	
Normal Flow	Actor 1. Pengguna mengakses <i>endpoint</i> pengingat obat dan tekanan darah dengan <i>access token</i> 3. Pengguna dapat melihat data dari <i>endpoint</i> pengingat obat dan tekanan darah	System 2. Sistem akan mengeksekusi fungsi melihat pengingat obat dan tekanan darah
Post Condition	1. Jika ada data baru, maka akan tampil di paling awal	

Use case pada Tabel 2 berawal ketika pengguna yang sudah *login* dengan *access token* mengakses sebuah *endpoint* khusus di sistem. Hal ini memastikan bahwa hanya pengguna yang sudah diverifikasi yang bisa menambah data baru, menjaga keamanan dan privasi informasi tersebut. Setelah mengakses *endpoint*, sistem akan menjalankan fungsi untuk membuat pengingat obat dan mencatat tekanan darah sesuai dengan data yang dimasukkan pengguna. Setelah berhasil, informasi baru ini akan disimpan di *database*, memastikan data pengguna tersimpan dengan aman dan mudah diakses lagi. Proses ini memberikan pengguna kontrol lebih dalam mengelola kesehatan mereka dan meningkatkan efektivitas pengingat yang telah diatur.

TABEL II
DESKRIPSI USE CASE DIAGRAM MEMBUAT PENGINGAT OBAT DAN TEKANAN DARAH

UC Name	Membuat Pengingat Obat & Tekanan Darah	
Actor	Pengguna	
Goal	Pengguna dapat membuat data pengguna	
Pre-Condition	1. Pengguna belum punya <i>access token</i>	
Normal Flow	Actor 1. Pengguna mengakses <i>endpoint</i> pengingat obat dan tekanan darah dengan <i>access token</i> 3. Pengguna dapat membuat data dari <i>endpoint</i> pengingat obat dan tekanan darah	System 2. Sistem akan mengeksekusi fungsi membuat pengingat obat dan tekanan darah
Post Condition	1. Jika fungsi sudah berjalan, maka data dibuat di <i>database</i>	

Use case pada Tabel 3 bermula saat pengguna yang sudah terverifikasi dengan *access token* masuk ke *endpoint* tertentu di sistem. Langkah ini penting untuk memastikan bahwa hanya pengguna yang berhak yang bisa mengedit data mereka, menjaga keamanan data tersebut. Setelah masuk, sistem akan memproses perubahan data pengingat obat atau tekanan darah sesuai dengan informasi terbaru yang dimasukkan oleh pengguna. Setelah proses edit selesai, data yang sudah diubah akan disimpan kembali ke dalam *database*. Ini memastikan bahwa setiap perubahan yang pengguna lakukan akan tercatat dengan benar dan bisa diakses lagi nanti. Dengan kemampuan untuk mengupdate data sendiri, pengguna punya kontrol lebih atas pengelolaan kesehatan mereka, memungkinkan mereka untuk menyesuaikan pengingat sesuai dengan kebutuhan atau perubahan kondisi kesehatan mereka.

Use case pada Tabel 4 dimulai ketika pengguna yang sudah memiliki *access token*, menunjukkan mereka telah *login*, mengakses *endpoint* khusus di sistem menggunakan *access token* tersebut. Selanjutnya, sistem akan menjalankan fungsi untuk menghapus data pengingat obat dan tekanan darah yang tidak lagi dibutuhkan. Pengguna

TABEL III
DESKRIPSI USE CASE DIAGRAM MENGUBAH PENGINGAT OBAT DAN TEKANAN DARAH

UC Name	Mengubah Pengingat Obat & Tekanan Darah	
Actor	Pengguna	
Goal	Pengguna dapat mengubah data pengguna	
Pre-Condition	1. Pengguna belum punya <i>access token</i>	
Normal Flow	Actor 1. Pengguna mengakses <i>endpoint</i> pengingat obat dengan <i>access token</i> 3. Pengguna dapat mengubah data dari <i>endpoint</i> pengingat obat dan tekanan darah	System 2. Sistem akan mengeksekusi fungsi mengubah pengingat obat dan tekanan darah
Post Condition	1. Jika fungsi sudah berjalan, maka data diubah di <i>database</i>	

bisa langsung menghapus data ini melalui *endpoint* terkait. Jika proses penghapusan berhasil, data yang terkait akan dihilangkan dari *database*. *Use case* ini dirancang untuk memastikan bahwa data yang tidak diperlukan atau diinginkan lagi oleh pengguna bisa dihapus dengan efisien, menjaga data dalam sistem tetap akurat dan relevan.

TABEL IV
DESKRIPSI USE CASE DIAGRAM MENGHAPUS PENGINGAT OBAT DAN TEKANAN DARAH

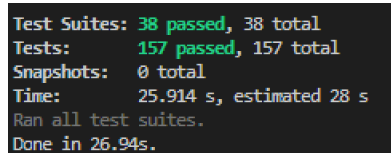
UC Name	Menghapus Pengingat Obat & Tekanan Darah	
Actor	Pengguna	
Goal	Pengguna dapat menghapus dari pengguna	
Pre-Condition	1. Pengguna belum punya <i>access token</i>	
Normal Flow	Actor 1. Pengguna mengakses <i>endpoint</i> pengingat obat dengan <i>access token</i> 3. Pengguna dapat menghapus data dari <i>endpoint</i> pengingat obat dan tekanan darah	System 2. Sistem akan mengeksekusi fungsi menghapus pengingat obat dan tekanan darah
Post Condition	1. Jika fungsi sudah berjalan, maka data dihapus di <i>database</i>	

III. HASIL DAN PEMBAHASAN

Dalam penyelidikan ini, sistem mengeksekusi serangkaian penilaian otomatis yang komprehensif untuk memastikan kinerja dan keandalan kerangka kerja aplikasi yang dikembangkan. Seperti yang diilustrasikan pada Gambar 6 dan 7, sistem telah melakukan total kumulatif 38 rangkaian pengujian yang mencakup 157 pengujian berbeda di berbagai komponen fungsional. Semua penilaian tersebut telah berhasil diselesaikan tanpa kegagalan yang teridentifikasi, menunjukkan bahwa setiap fungsionalitas aplikasi beroperasi sesuai dengan standar yang diantisipasi. Evaluasi dilakukan pada beberapa modul penting, termasuk otentikasi pengguna (auth), pengelolaan data tekanan darah, dan pengawasan pengingat obat. Pengujian tambahan diperluas untuk mencakup modul video, artikel, bookmark, jadwal, dan kuesioner.

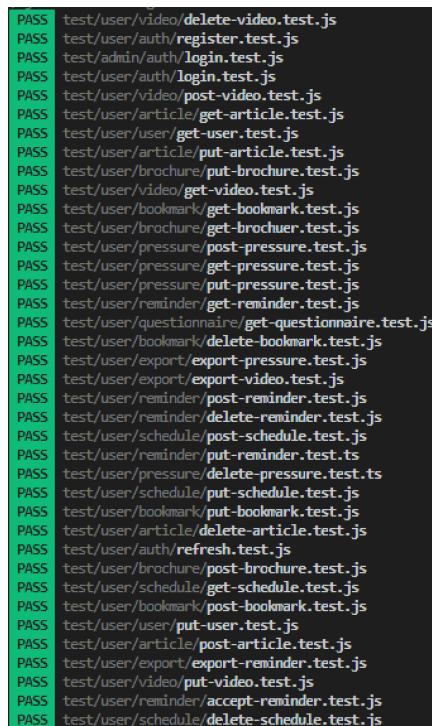
Dalam modul autentikasi, peneliti ini menguji fungsionalitas pendaftaran (*register*), masuk (*login*), dan pembaruan token (*refresh token*). Modul tekanan darah meliputi pengujian terhadap penambahan (*post*), pengambilan (*get*), pembaruan (*put*), dan penghapusan (*delete*) data tekanan. Sementara itu, modul pengingat mencakup pengujian terhadap penambahan, pengambilan, pembaruan, penghapusan, dan penerimaan pengingat. Keberhasilan semua tes ini menandakan bahwa semua fitur inti dari aplikasi sudah memenuhi kriteria yang diharapkan.

Secara keseluruhan, dapat dinyatakan secara meyakinkan bahwa seluruh sistem secara mahir dan mulus mengeksekusi prosedur pengujian komprehensif tanpa menghadapi masalah atau gangguan yang signifikan, dengan total durasi yang diperlukan untuk menyelesaikan pelaksanaan semua rangkaian pengujian sebesar 25.914 detik yang mengesankan, sementara waktu pemrosesan keseluruhan dicatat hanya 28 detik. Hasil yang menarik ini dengan tegas menunjukkan bahwa sistem menunjukkan kecepatan dan efisiensi yang luar biasa dalam konteks



```
Test Suites: 38 passed, 38 total
Tests: 157 passed, 157 total
Snapshots: 0 total
Time: 25.914 s, estimated 28 s
Ran all test suites.
Done in 26.94s.
```

Gambar. 6. Total Pengujian dan Durasi



```
PASS test/user/video/delete-video.test.js
PASS test/user/auth/register.test.js
PASS test/admin/auth/login.test.js
PASS test/user/auth/login.test.js
PASS test/user/video/post-video.test.js
PASS test/user/article/get-article.test.js
PASS test/user/user/get-user.test.js
PASS test/user/article/put-article.test.js
PASS test/user/brochure/put-brochure.test.js
PASS test/user/video/get-video.test.js
PASS test/user/bookmark/get-bookmark.test.js
PASS test/user/brochure/get-brochure.test.js
PASS test/user/pressure/post-pressure.test.js
PASS test/user/pressure/get-pressure.test.js
PASS test/user/pressure/put-pressure.test.js
PASS test/user/reminder/get-reminder.test.js
PASS test/user/questionnaire/get-questionnaire.test.js
PASS test/user/bookmark/delete-bookmark.test.js
PASS test/user/export/export-pressure.test.js
PASS test/user/export/export-video.test.js
PASS test/user/reminder/post-reminder.test.js
PASS test/user/reminder/delete-reminder.test.js
PASS test/user/schedule/post-schedule.test.js
PASS test/user/reminder/put-reminder.test.js
PASS test/user/pressure/delete-pressure.test.js
PASS test/user/schedule/put-schedule.test.js
PASS test/user/bookmark/put-bookmark.test.js
PASS test/user/article/delete-article.test.js
PASS test/user/auth/refresh.test.js
PASS test/user/brochure/post-brochure.test.js
PASS test/user/schedule/get-schedule.test.js
PASS test/user/bookmark/post-bookmark.test.js
PASS test/user/user/put-user.test.js
PASS test/user/article/post-article.test.js
PASS test/user/export/export-reminder.test.js
PASS test/user/video/put-video.test.js
PASS test/user/reminder/accept-reminder.test.js
PASS test/user/schedule/delete-schedule.test.js
```

Gambar. 7. Hasil dari pengujian tekanan darah dan pengingat obat

metodologi pengujian otomatis. Gambar 8 menggambarkan laporan cakupan kode yang mengevaluasi metrik seperti pernyataan, percabangan, fungsi, dan cakupan garis selama pengujian. Laporan ini sangat penting untuk

menentukan kualitas dan keandalan kode sumber. `app.js` menunjukkan cakupan pernyataan 90,9%, cakupan percabangan 100%, cakupan fungsi 100%, dan cakupan baris 90,9%, dengan 25 baris kode yang tidak memiliki cakupan pengujian. `BloodPressure.controller.js` menunjukkan cakupan 100% di seluruh pernyataan, percabangan, dan fungsi, dengan hanya 1 baris yang terungkap, mencerminkan kinerja pengujian yang luar biasa. `reminder.controller.js` menyajikan cakupan pernyataan 95,58%, cakupan percabangan 93,33%, cakupan fungsi 100%, dan cakupan garis 95,23%, dengan garis yang tidak tertutup diidentifikasi pada nomor 156, 164, dan 192. Berbagai modul selain dari blood pressure dan reminder memberikan bukti kuat bahwa terintegrasi secara komprehensif ke dalam rangka *unit testing*, yang mulai dengan pengujian `article`, `bookmark`, `export`, `questionnaire`, `userDashboard`, `authentication`, dan `routes` yang telah dijabarkan.

File	% Stats	% Branch	% Funcs	% Lines	Uncovered Line #s
mahuti-backend	91.66	100	0	91.66	
app.js	91.66	100	0	91.66	28
auth.controller.js	89.47	83.33	100	89.47	50-57
article.controller.js	95	89.28	100	94.64	9,15,32
bloodPressure.controller.js	98.27	96.87	100	98.11	40
bookmark.controller.js	84.9	62.5	100	89.58	15,26,54,78,90
export.controller.js	80	50	100	88.88	24,47,70
questionnaire.controller.js	100	100	100	100	
reminder.controller.js	95.58	93.33	100	95.23	156,164,192
userDashboard.controller.js	100	100	100	100	
mahuti-backend/app/routes	100	100	100	100	
routes.js	100	100	100	100	
mahuti-backend/app/routes/admin	100	100	100	100	
auth.route.js	100	100	100	100	
bloodPressure.route.js	100	100	100	100	
bookmark.route.js	100	100	100	100	
brochure.route.js	100	100	100	100	
export.route.js	100	100	100	100	
index.js	0	0	0	0	
questionnaire.route.js	100	100	100	100	
questionnaireQuestion.route.js	100	100	100	100	
reminder.route.js	100	100	100	100	
schedule.route.js	100	100	100	100	
user.route.js	100	100	100	100	
video.route.js	100	100	100	100	

Gambar. 8. Laporan Hasil Pengujian

A. Implementasi Pengujian Unit Peningkat Obat *Service*

Pengujian fungsi `createReminder` pada Gambar 9 ini memastikan bahwa semua data yang diperlukan disertakan sebelum penyimpanan ke *database*, seperti terlihat dari pengecekan kondisional pada baris 17-26 yang

mengembalikan *error* jika data tidak lengkap. Setelah verifikasi, fungsi menggunakan Prisma ORM untuk membuat *record* baru, meminimalisir potensi kesalahan data.

```

1 import { prisma } from "../../lib/dbConnect.js";
2 import { verifyToken } from "../../lib/tokenHandler.js";
3
4 12x export const createReminder = async (req, res, next) => {
5 3x   try {
6     const {
7       user_id,
8       medicine_name,
9       medicine_taken,
10      medicine_total,
11      amount,
12      cause,
13      cap_size,
14      medicine_time,
15 3x    } = req.body;
16
17 3x    if (
18      !user_id ||
19      !medicine_name ||
20      !medicine_taken ||
21      !medicine_total ||
22      !amount ||
23      !cause ||
24      !cap_size ||
25      !medicine_time
26    ) {
27 1x      return res.status(400).json({
28        status: 400,
29        message: "Please provide all required fields.",
30      });
31    }
32
33 2x    const newReminder = await prisma.reminders.create({
34      data: {
35        user_id,
36        medicine_name,
37        medicine_taken,
38        medicine_total,
39        amount,
40        cause,
41        cap_size,
42        medicine_time,
43      },
44    });
45
46 1x    res.status(201).json({ success: true, data: newReminder });
47  } catch (error) {
48 1x    next(error);
49  }
50 };
    
```

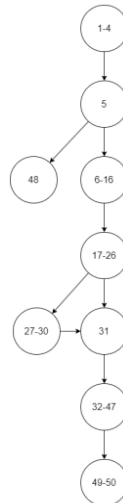
Gambar. 9. Pengujian Unit Peningkat Obat

Pada Tabel 5 TC-01, fungsi ini memeriksa apakah semua field seperti *user_id*, *medicine_name*, *medicine_taken*, *medicine_total*, *amount*, *cause*, *cap_size*, dan *medicine_time* telah diisi. Jika salah satu dari field ini kosong akan memberikan respons 400. Hasil dari tes ini menunjukkan bahwa fungsi tersebut berhasil menjalankan semua perintah yang diharapkan, dengan *statement coverage* mencapai 100%, menandakan bahwa semua bagian dari kode telah diuji secara efektif dan sesuai dengan ekspektasi.

TABEL V
 PENGUJIAN STATEMENT COVERAGE PADA PENGINGAT OBAT

Test Case ID	Jalur	Output	Ket.	Statement Tereksekusi	Statement Coverage
TC-01	if (!user_id !medicine_name !medicine_taken !medicine_total !amount !cause !cap_size !medicine_time) { return res.status(400).json({ status: 400, message: "Please provide all required fields.", }); }	Mengembalikan data pesan "Please provide all required fields."	Berhasil	54	54/54 = 100%

Menurut *flowgraph* yang disajikan pada Gambar 10 terdapat total 50 *statement*, 4 *branch*, 9 *node* (N), dan 9 *edge* (E).



Gambar. 10. Flowgraph Pengingat Obat

B. Implementasi Pengujian Unit Tekanan Darah *Service*

Fungsi *createBloodPressure* dapat dilihat pada Gambar 11 dirancang untuk memvalidasi dan menyimpan data tekanan darah, yang meliputi *sistol*, *diastol*, dan detak jantung, ke dalam *database* menggunakan Prisma ORM. Sebelum penyimpanan, fungsi ini memeriksa keabsahan *access token* akses melalui fungsi *verifyToken* dan mengembalikan respons sesuai status yang diperoleh. Selanjutnya, memastikan bahwa nilai *sistol*, *diastol*, dan detak jantung berada dalam rentang yang ditentukan (20 hingga 200) dan mengharuskan input berupa bilangan bulat. Jika nilai yang diberikan tidak memenuhi kriteria ini, fungsi tersebut mengembalikan status HTTP 400 dengan pesan kesalahan yang relevan. Setelah validasi berhasil, data diolah dan disimpan, dan respons sukses dikembalikan dengan status HTTP 201. Pendekatan ini meningkatkan keandalan dan keakuratan data tekanan darah yang disimpan, sekaligus memastikan integritas input yang diterima oleh sistem.

```

1  import { prisma } from "../../lib/dbConnect.js";
2  import { verifyToken } from "../../lib/tokenHandler.js";
3
4  12x export const createBloodPressure = async (req, res, next) => {
5  15x   try {
6  15x     const { sistol, diastole, heartbeat } = req.body;
7  15x     const data = verifyToken(req.headers.access_token);
8  15x     if (data?.status) return res.status(data.status).json(data);
9
10  13x     const image = req.file ? req.file.path : "";
11
12  13x     if (parseInt(sistol) < 20 || parseInt(sistol) >= 200) {
13  5x       return res.status(400).json({
14         status: 400,
15         message: "Sistol value should be between 20 and 200.",
16       });
17  8x     } else if (parseInt(diastole) < 20 || parseInt(diastole) >= 200) {
18  2x       return res.status(400).json({
19         status: 400,
20         message: "Diastole value should be between 20 and 200.",
21       });
22  6x     } else if (parseInt(heartbeat) < 20 || parseInt(heartbeat) >= 200) {
23  2x       return res.status(400).json({
24         status: 400,
25         message: "Heartbeat value should be between 20 and 200.",
26       });
27  4x     } else if (
28       sistol.includes(",") ||
29       diastole.includes(",") ||
30       heartbeat.includes(",") ||
31       sistol.includes(".") ||
32       diastole.includes(".") ||
33       heartbeat.includes(".")
34     ) {
35  1x       return res.status(400).json({
36         status: 400,
37         message: "Please use whole numbers for blood pressure values.",
38       });
39     }
40
41  2x     const newBloodPressure = await prisma.blood_pressures.create({
42       data: {
43         user_id: data.id,
44         image,
45         sistol: parseInt(sistol),
46         diastole: parseInt(diastole),
47         heartbeat: parseInt(heartbeat),
48       },
49     });
50  2x     res.status(201).json({ success: true, data: newBloodPressure });
51   } catch (error) {
52  1x     next(error);
53   }
54 }

```

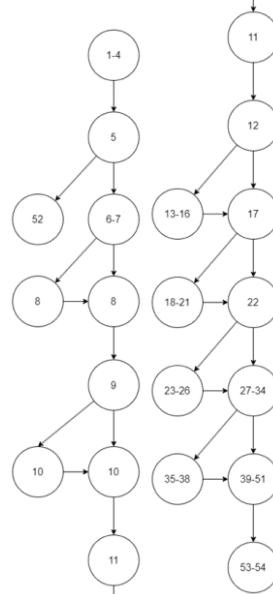
Gambar. 11. Pengujian Unit Tekanan Darah

Pada Tabel 6 TC-02, fungsi memeriksa apakah nilai sistolik kurang dari 20 atau lebih dari atau sama dengan 200. Jika kondisi ini terpenuhi, fungsi mengembalikan status *error* 400 dan pesan bahwa nilai harus antara 20 dan 200. Hasil tes ini menunjukkan bahwa fungsi berjalan dengan sukses, mencapai 100% *statement coverage*, yang berarti semua bagian kode diuji dan berfungsi sesuai dengan harapan.

TABEL VI
 PENGUJIAN STATEMENT COVERAGE PADA TEKANAN DARAH

Test Case ID	Jalur	Output	Ket.	Statement Tereksekusi	Statement Coverage
TC-02	<pre> if (parseInt(sistol) < 20 parseInt(sistol) >= 200) { return res.status(400).json({ status: 400, message: "Sistol value should be between 20 and 200.", }); }</pre>	Mengembalikan data Berhasil pesan "Sistol value should be between 20 and 200."		50	50/50= 100%

Berdasarkan flowgraph pada Gambar 12 terdapat total 54 statement, 14 branch, 21 node (N), dan 26 edge (E). Perbandingan hasil penelitian ini dengan penelitian sebelumnya yaitu pada penelitian sebelumnya metode yang digunakan masih melalui pendekatan *branch coverage* dengan penekanan pada tes berbasis pencarian seperti



Gambar. 12. Flowgraph Tekanan Darah

testability transformations, *system*, dan *integration testing* [21]. Sedangkan penelitian ini menggunakan pendekatan *branch coverage* dan *statement coverage*. Selain itu, perbandingan lainnya yaitu pada penelitian sebelumnya pengujian menggunakan junit dengan menggunakan pola lebih berfokus pengkodean yang baik [22]. Sedangkan, pada penelitian ini menggunakan cakupan kode dengan melihat metrik *statement*, *branchm function*, *line*, dan *uncovered lines*. Selanjutnya pada penelitian terakhir menggunakan metode *branch coverage* dengan *machine learning* untuk menyelidiki faktor-faktor yang paling memengaruhi akurasi prediksi [23]. Sedangkan pada penelitian ini menggunakan *library* Express.js dengan fleksibilitas pada pengujian *unit testing*.

IV. KESIMPULAN

Penelitian ini membuktikan bahwa penggunaan metode *White-Box Testing* dengan teknik *Branch Coverage* dan *Statement Coverage* sangat efektif untuk meningkatkan kualitas dan keandalan aplikasi. Hal ini dibuktikan dengan hasil pengujian aplikasi MAHATI yang berhasil menjalankan seluruh 12 *suite* pengujian dengan total 65 tes individu tanpa mengalami kegagalan. Ini menunjukkan bahwa setiap fungsi aplikasi bekerja sesuai dengan harapan, dan membuktikan bahwa proses pengujian yang dilakukan sangat efektif. Dengan demikian, metode *white box testing* terbukti ampuh dalam meningkatkan kualitas perangkat lunak, dan dapat diaplikasikan untuk meningkatkan kualitas aplikasi lain di masa depan.

V. SARAN

Untuk penelitian di masa depan, ada beberapa hal yang bisa dilakukan untuk meningkatkan kualitas dan kedalaman studi. Selain *White-Box Testing*, integrasikan metode pengujian lain seperti *Black-Box Testing* atau *Grey-Box Testing* untuk melihat fungsionalitas dan interaksi antar sistem secara menyeluruh. Pastikan keamanan dan performa aplikasi dengan melakukan *penetration* dan *load testing*. Setelah pengujian, tinjau dan analisa kode untuk menemukan potensi peningkatan, dan *refactor* kode untuk meningkatkan kejelasan, efisiensi, dan kemudahan pengujian. Dengan demikian, penelitian selanjutnya diharapkan dapat lebih kuat, menyeluruh, dan menghasilkan aplikasi yang lebih berkualitas tinggi.

VI. PENGAKUAN

Ucapan terima kasih kepada Badan Riset dan Inovasi Nasional (BRIN) atas bantuan pendanaan penelitian ini dalam program *Research and Innovation Funding for Advanced Indonesia (RIIM)* bekerja sama dengan Lembaga Pengelolaan Dana Pendidikan (LPDP) (NOMOR 37//II.7/HK/2023)

DAFTAR PUSTAKA

- [1] J. Tian, *Software Quality Engineering*. 2005. [Online]. Available: <https://onlinelibrary.wiley.com/doi/book/10.1002/0471722324>
- [2] V. Garousi, A. Rainer, P. Lauvås, and A. Arcuri, "Software-testing education: A systematic literature mapping," *Journal of Systems and Software*, vol. 165, p. 110570, Jul. 2020, doi: 10.1016/j.jss.2020.110570.
- [3] J. L. Min, A. Istiqomah, and A. Rahmani, "Evaluasi Penggunaan Manual dan Automated Software Testing pada Pelaksanaan End-To-End Testing," *JIT*, vol. 6, no. 1, p. 18, Apr. 2020, doi: 10.31884/jtt.v6i1.256.
- [4] J. E. Cook and A. L. Wolf, "Software Process Validation: Quantitatively Measuring the Correspondence of a Process to a Model," *ACM Trans. Softw. Eng. Methodol.*, vol. 8, no. 2, pp. 147–176, Apr. 1999, doi: 10.1145/304399.304401.
- [5] S. Bardin, O. Chebaro, M. Delahaye, and N. Kosmatov, "An All-in-One Toolkit for Automated White-Box Testing," in *Tests and Proofs*, vol. 8570, M. Seidl and N. Tillmann, Eds., in Lecture Notes in Computer Science, vol. 8570, Cham: Springer International Publishing, 2014, pp. 53–60. doi: 10.1007/978-3-319-09099-3_4.
- [6] Y. Purwananto, A. Bramantoro, and L. Harini, "Perancangan dan Pembuatan Case Tool Software Testing Menggunakan Metode Static Data Flow Analysis (Studi Kasus Program Berbasis C++)," *JUTI*, vol. 4, no. 1, p. 60, Jan. 2005, doi: 10.12962/j24068535.v4i1.a247.
- [7] S. M. Sohan, F. Maurer, C. Anslow, and M. P. Robillard, "A study of the effectiveness of usage examples in REST API documentation," in *2017 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, Raleigh, NC: IEEE, Oct. 2017, pp. 53–61. doi: 10.1109/VLHCC.2017.8103450.
- [8] A. A. Prayogi, M. Niswar, Indrabayu, and M. Rijal, "Design and Implementation of REST API for Academic Information System," *IOP Conf. Ser.: Mater. Sci. Eng.*, vol. 875, no. 1, p. 012047, Jun. 2020, doi: 10.1088/1757-899X/875/1/012047.
- [9] A. Mardan, "Using Express.js 4 to Create Node.js Web Apps," in *Practical Node.js: Building Real-World Scalable Web Apps*, A. Mardan, Ed., Berkeley, CA: Apress, 2014, pp. 33–56. doi: 10.1007/978-1-4302-6596-2_2.
- [10] S. A. B. Cahyono, S. Sucipto, and R. Firlana, "Implementasi Otentikasi Website Node JS Express Menggunakan Passport," *jsitik*, vol. 2, no. 1, pp. 33–40, Nov. 2023, doi: 10.53624/jsitik.v2i1.309.
- [11] R. Lisgiani and S. Nurmajid, "Implementasi Autentikasi Dari Sisi Backend pada Arsitektur Microservices Menggunakan Express JS," *infotronik*, vol. 7, no. 1, p. 27, Jun. 2022, doi: 10.32897/infotronik.2022.7.1.1330.
- [12] N. Nasrul and A. Izhar, "Pengembangan REST API dengan menggunakan Express JS untuk mencari Mentor Pribadi," *j. inform. terpadu*, vol. 9, no. 2, pp. 92–102, Sep. 2023, doi: 10.54914/jit.v9i2.974.
- [13] H. Yaapa, *Express Web Application Development*. Packt Publishing, 2013.
- [14] H. Suryotrisongko, D. P. Jayanto, and A. Tjahyanto, "Design and Development of Backend Application for Public Complaint Systems Using Microservice Spring Boot," *Procedia Computer Science*, vol. 124, pp. 736–743, 2017, doi: 10.1016/j.procs.2017.12.212.
- [15] R. Nicoara, "The Backend," in *How to be a Web Developer: A Complete Beginner's Guide on What to Know and Where to Start*, R. Nicoara, Ed., Berkeley, CA: Apress, 2023, pp. 75–92. doi: 10.1007/978-1-4842-9663-9_5.
- [16] L. Portella, "Backend: What's Underwater," in *A Friendly Guide to Software Development: What You Should Know Without Being a Developer*, L. Portella, Ed., Berkeley, CA: Apress, 2023, pp. 93–112. doi: 10.1007/978-1-4842-8969-3_6.
- [17] L. Özdamar and E. Alanya, "Uncertainty Modelling in Software Development Projects (With Case Study) | Annals of Operations Research." Accessed: Oct. 06, 2024. [Online]. Available: <https://link.springer.com/article/10.1023/A:1010906116000>
- [18] M. Fazzini, "Automatic Detection and Correction of Client-Side and Server-Side Input-Validation Inconsistencies," University of Illinois at Chicago, 2012.
- [19] T. Hagos, "Unit Testing," in *Beginning Kotlin: Build Applications with Better Code, Productivity, and Performance*, T. Hagos, Ed., Berkeley, CA: Apress, 2023, pp. 185–199. doi: 10.1007/978-1-4842-8698-2_9.
- [20] D. Honfi and Z. Micskei, "Automated Isolation for White-Box Test Generation," *Information and Software Technology*, vol. 125, p. 106319, Sep. 2020, doi: 10.1016/j.infsof.2020.106319.
- [21] P. Derakhshanfar, X. Devroey, and A. Zaidman, "Basic Block Coverage for Search-Based Unit Testing and Crash Reproduction," *Empir Software Eng*, vol. 27, no. 7, p. 192, Dec. 2022, doi: 10.1007/s10664-022-10155-0.
- [22] D. D. Ma'ayan, "The Quality of Junit Tests: an Empirical Study Report," in *Proceedings of the 1st International Workshop on Software Qualities and Their Dependencies*, Gothenburg Sweden: ACM, May 2018, pp. 33–36. doi: 10.1145/3194095.3194102.
- [23] G. Grano, T. V. Titov, S. Panichella, and H. C. Gall, "Branch Coverage Prediction in Automated Testing," *J Software Evolu Process*, vol. 31, no. 9, p. e2158, Sep. 2019, doi: 10.1002/smr.2158.
- [24] J. Park, S. An, D. Youn, G. Kim, and S. Ryu, "JEST: N+1-Version Differential Testing of Both JavaScript Engines and Specification," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, Madrid, ES: IEEE, May 2021, pp. 13–24. doi: 10.1109/ICSE43902.2021.00015.
- [25] R. Yunisa and H. P. Putro, "Perbandingan 2 Teknik White Box Testing: Statement Coverage dan Branch Coverage Testing," 2019.
- [26] D. Fisman and G. Rosu, Eds., *Maximizing Branch Coverage with Constrained Horn Clauses*, vol. 13244. in Lecture Notes in Computer Science, vol. 13244. Cham: Springer International Publishing, 2022. doi: 10.1007/978-3-030-99527-0.
- [27] A. Parsai and S. Demeyer, "Comparing Mutation Coverage Against Branch Coverage in an Industrial Setting," *Int J Softw Tools Technol Transfer*, vol. 22, no. 4, pp. 365–388, Aug. 2020, doi: 10.1007/s10009-020-00567-y.
- [28] H. Nurfauziah and I. Jamaliyah, "Perbandingan Metode Testing Antara Blackbox dengan Whitebox pada Sebuah Sistem Informasi," vol. 8, no. 2, 2022.
- [29] J. Paul C., *Software Testing: a Craftsman's Approach*. CRC press, 2018.
- [30] J. C. King, "Symbolic Execution and Program Testing," *Commun. ACM*, vol. 19, no. 7, pp. 385–394, Jul. 1976, doi: 10.1145/360248.360252.

- [31] Y. Andriyani, I. Daqiqil Id, E. Mahdiyah, and A. Aminuddin, "Use Case Realization in Software Reverse Engineering," *ISI*, vol. 27, no. 2, pp. 335–341, Apr. 2022, doi: 10.18280/isi.270218.